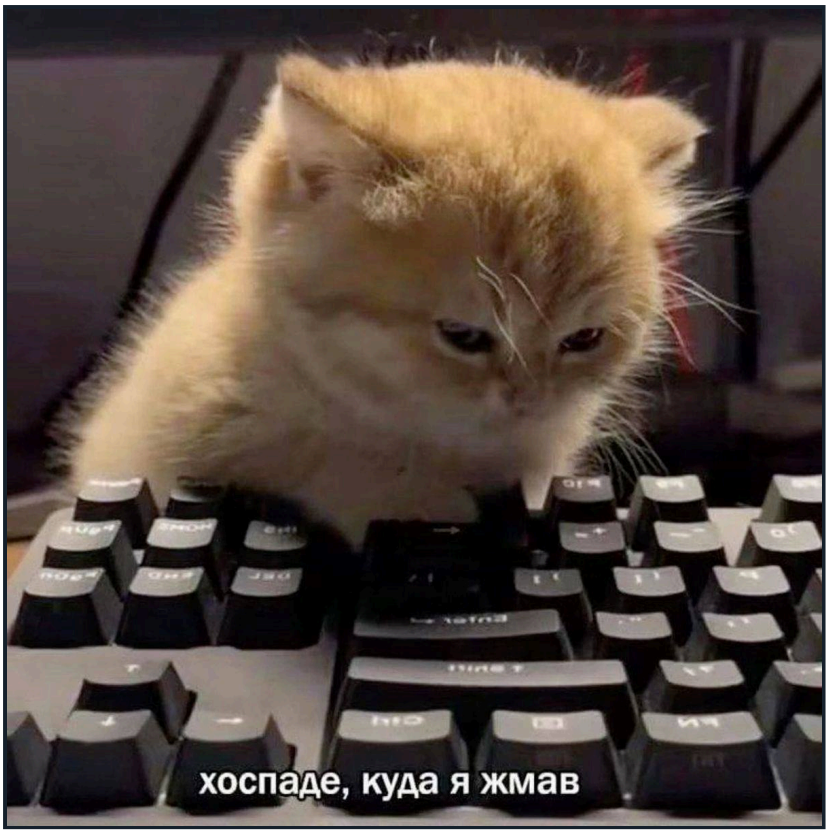
Домашнее задание № 13 · Нормализация и частоты

ОП.04 «Основы алгоритмизации и программирования», 3 курс.

Максим Николаевич не хочет задавать вам домашнее задание. Но очень хочет.



Что нужно сделать

1. **Довести решения занятия.** `solution.py` проходит все 36 публичных тестов. `bench_results.txt` — вывод `python bench.py` на вашей машине. `complexity_note.md` заполнен по трём функциям: правила нормализации и кто их задаёт, что хранится в словаре, время, память и таблицы двух опытов замера.

2. **Три ошибки на границе.** В `bugs_13.py` три функции: `is_pangram_ru`, `top_letter`, `all_unique`. Каждая верно отвечает на обычных входах и ошибается на граничном. Для каждой: найдите такой вход, запишите в `tests_bugs_13.py` падающий тест, исправьте одну строку и прогоните тесты снова. Вывод pytest сохраните в `bugs_run.txt`, в `complexity_note.md` допишите три строки «функция → вход → что было нарушено».

3. **Все анаграммы в строке** (LeetCode 438). Напишите в `solution.py` функцию `find_anagrams(s, p)`. Вернуть список индексов начала всех подстрок `s`, которые являются анаграммами `p`, по возрастанию. Символы сравниваются как есть, с регистром, без нормализации. Пример: `s = 'cbaebabacd'`, `p = 'abc'` → `[0, 6]`; `s = 'abab'`, `p = 'ab'` → `[0, 1, 2]`. Время O(n · k), где n — длина `s`, k — число разных символов; пересчитывать частоты каждой подстроки заново нельзя. Подсказка: окно фиксированной длины из модуля 9 книги о массивах и частотный словарь этого занятия. В `complexity_note.md` объясните, что

хранится в окне и как оно меняется при сдвиге. Не меньше шести своих тестов, среди них `p` длиннее `s`, вхождения, которые перекрываются, повторяющиеся буквы в `p` (`'aaaaa'`, `'aa'`), кириллица и случай, когда вхождений нет.

Тест к `bugs_13.py` засчитывается, если до исправления он падает, а после — проходит. Тест, который проходит и на ошибочной версии, ошибку не находит.

Как сдавать

Файлы	`portfolio/lesson_13/`: `solution.py`, `tests.py`, `bench_results.txt`, `complexity_note.md`, `initial_attempt.txt`, `bugs_13.py`, `tests_bugs_13.py`, `bugs_run.txt`
Ветка и PR	Ветка `hw-13`, коммиты с содержательными сообщениями, Pull Request в собственную `main`
Срок	До начала занятия 14
Проверяем	Зелёный pytest по всем файлам; три теста к `bugs_13.py`, которые падают до исправления; шесть тестов к `find_anagrams`; заметка без пустых пунктов
ИИ	Код и тесты пишете сами: на контроле занятия вы объясняете, какие символы значимы и почему, и сравниваете сортировку со словарём своими словами. Оформление `.md` и редактуру формулировок можно поручить модели. **Я сам так делаю всегда, поэтому и вам запрещать не буду.**